

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Last week, this week.

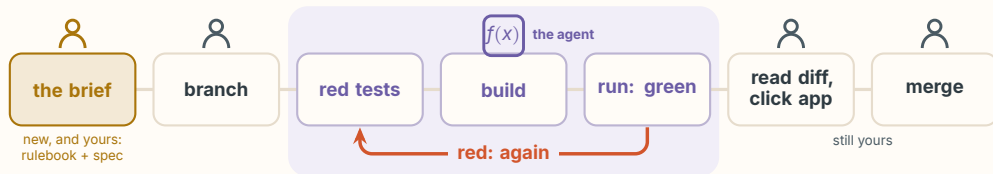
LAST WEEK

you, at every station



TODAY

same stations; three of them run without you



Same stations. Three now run without you. You write the brief instead.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Delegating: the brief, the judge, the loop
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

Why git exists.



"I'm an egotistical bastard, and I name all my projects after myself. First Linux, now git."

- 2005. The Linux kernel: thousands of people mailing changes to one man — and the commercial tool that tracked them revoked its free licence.
- He wrote git himself, in about ten days, and was running the kernel on it two months later.
- Everyone holds the **whole history**. Every change is named by its contents, so nothing can be altered quietly. And it is **fast**.

main

feature 1

main

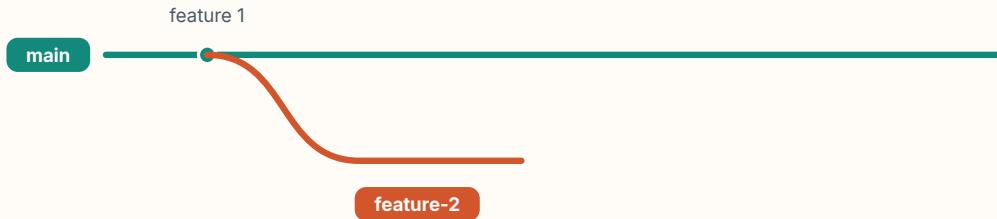


One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.

```
$ git show
```

```
diff --git a/logic.py b/logic.py
+def balances(group_id):
+    return paid(group_id) - shares(group_id)
```

A branch.

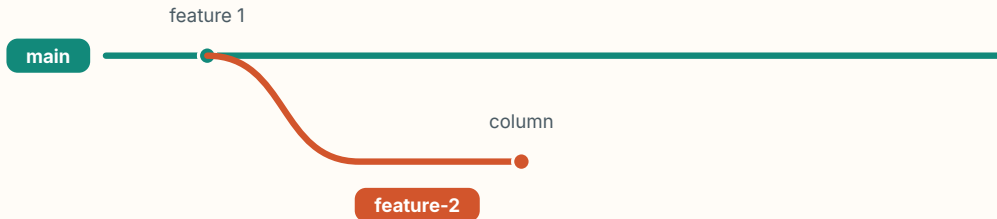


A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

```
$ git diff main feature-2
```

no output — a new branch is an identical copy

A commit.

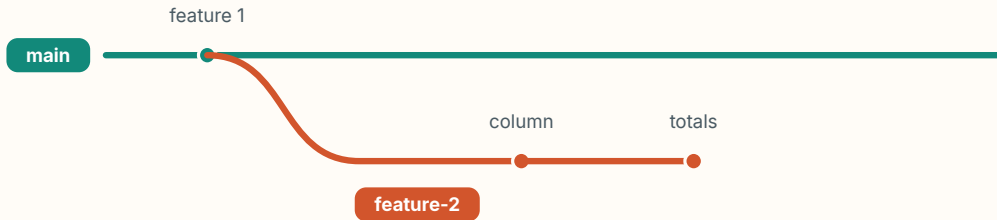


A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

```
$ git show
```

```
diff --git a/db.py b/db.py
    COLUMNS = ["id", "payer", "amount",
-           "date"]
+           "date", "category"]
```

Working on a branch.

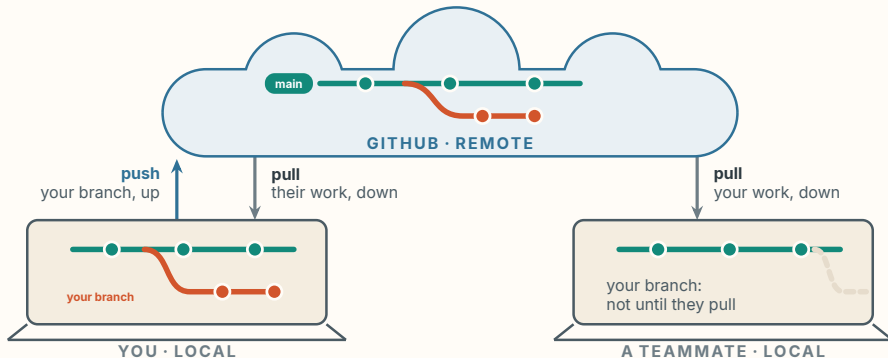


Two commits in, the tests are still red, the feature is half-built. Everyone else's **main** is untouched.

```
$ git diff main feature-2
```

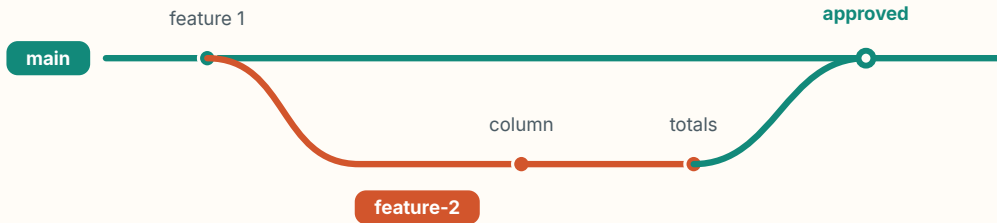
```
diff --git a/db.py b/db.py
-         "date"]
+         "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

Local, and remote.



One copy in the cloud, one on each laptop, and nothing crosses by itself. **Push** sends yours up, **pull** brings theirs down. A **pull request** asks someone to read it before **main** takes it.

Merge.

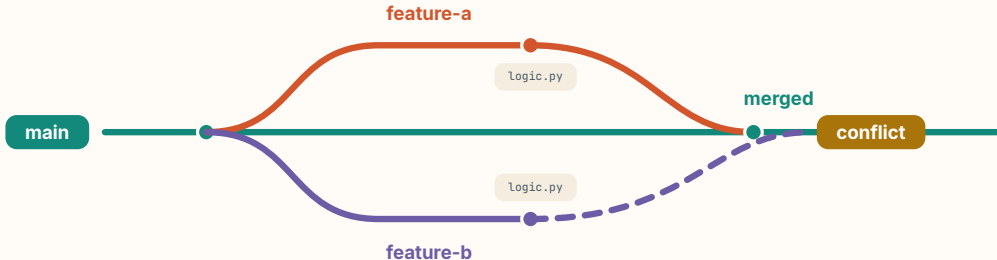


Merge is the sign-off. **main** now has your work, and the next person starts from there.

```
$ git diff main~1 main
```

```
diff --git a/db.py b/db.py
-      "date"]
+      "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

A conflict.



Different files, git merges by itself.
When two branches change **the same lines**, it stops and asks which version is right.

```
$ cat logic.py
```

```
<<<<<< main
    return sums_by("category", group_id)
=====
    return totals_by_category(group_id)
>>>>>> feature-b
```

The record.

HISTORY



Merge feature-2: categories

c41d8e2

you · today, 10:42

Add the totals

7d21e0b

you · yesterday, 17:05

Add the category column

4c8ab77

you · yesterday, 14:02

Feature 1: balances

91e04d5

you · 15 Sep

4c8ab77

Add the category column · you

go back to here

db.py

COLUMNS = ["id", "payer", "amount",

- "date"]

+ "date", "category"]

1 file changed · 1 line

Every commit on record: **what** changed, **who** did it, **when**. Open any one and read its diff — or go back to it.

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

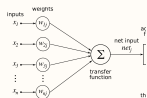
A model that can only write text — and the software around it that lets it act.

AI research chronology.

Neuron model

1958

Perceptron
learns from
examples



AI Winter

1969

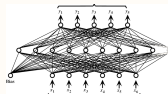
Perceptrons
one layer cannot
do XOR



New hope for neural networks

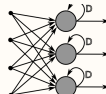
1986

Backpropagation
many layers,
trainable at last



1990-97

RNN
memory
across a
sequence



Scale

2017

Transformer
every word
looks at every
other, in parallel



The ideas were all there. What research still needed was **compute** and **data**, and both arrived after 2007.

Rosenblatt (1958) · Minsky & Papert (1969) · Rumelhart, Hinton & Williams (1986) · Elman (1990) · Hochreiter & Schmidhuber (1997) · Vaswani et al. (2017)

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

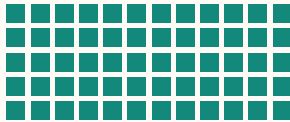
Gaming brought cheap parallel arithmetic.

A CPU

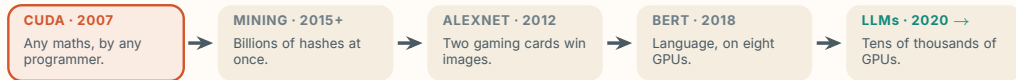


a few, fast · one thing each

A GPU



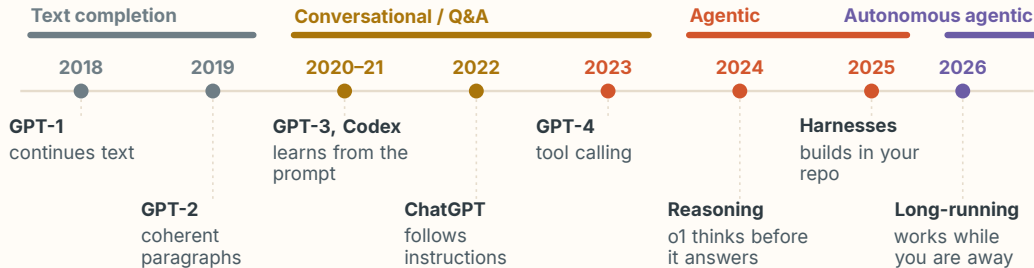
thousands, slower · all at once



NVIDIA made GPUs for gamers, but CUDA opened them for general-purpose computing. Training a neural network is nothing but parallel matrix arithmetic.

CUDA 1.0 (2007) · Krizhevsky et al. (2012), AlexNet on two GTX 580s · Devlin et al. (2018), BERT

2018 onwards.



GPT-2 writes a paragraph.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

Guess the next word.

1 · HIDE A WORD

The scientist named the population,
after their distinctive _____

2 · GUESS

$f(x)$

horn 62%
colour 14%
feature 9%
size 5%

3 · CORRECT

the page says: horn

off by a bit?
get corrected

next word, next page · a few billion times

That is all training is: hide the next word, guess, get corrected.

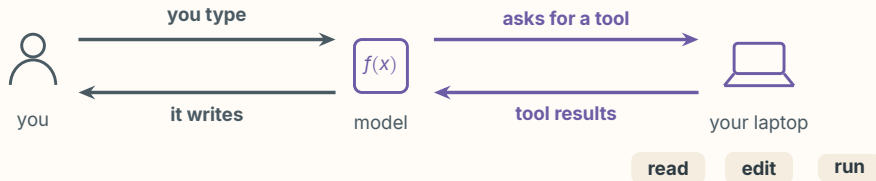
Trained on 40 GB of text: 8 million pages Reddit users had linked to · Radford et al. (2019), *Language Models are Unsupervised Multitask Learners*

A language model.



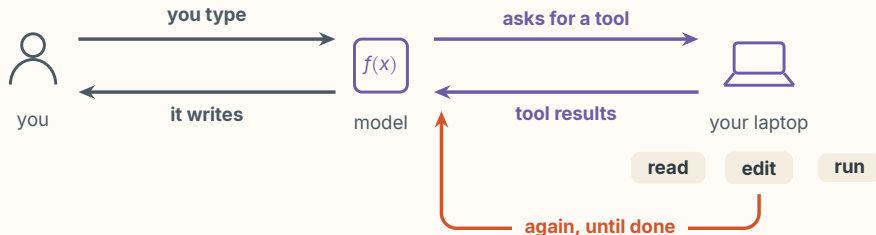
Text in, text out. It cannot open a file or run anything: you copy in, you paste out, and **you** are the one who finds out whether it works.

Tools.



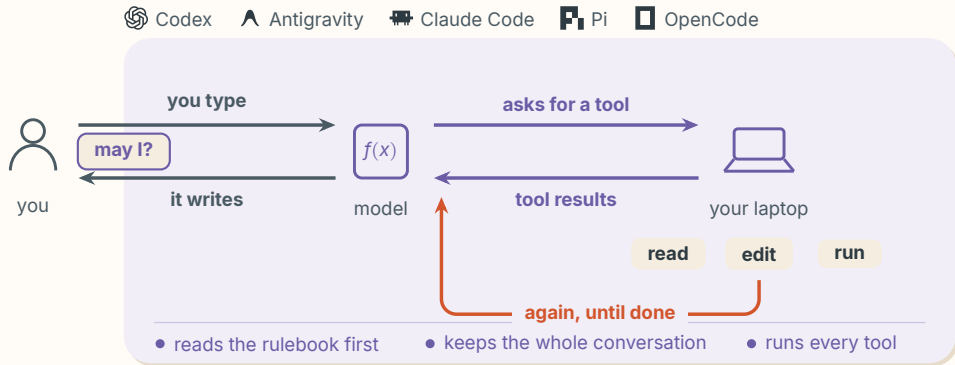
A **tool** is an action the model can request but cannot carry out itself: read a file, edit a file, run a command. Tool results are shown to **the model**.

An agent.



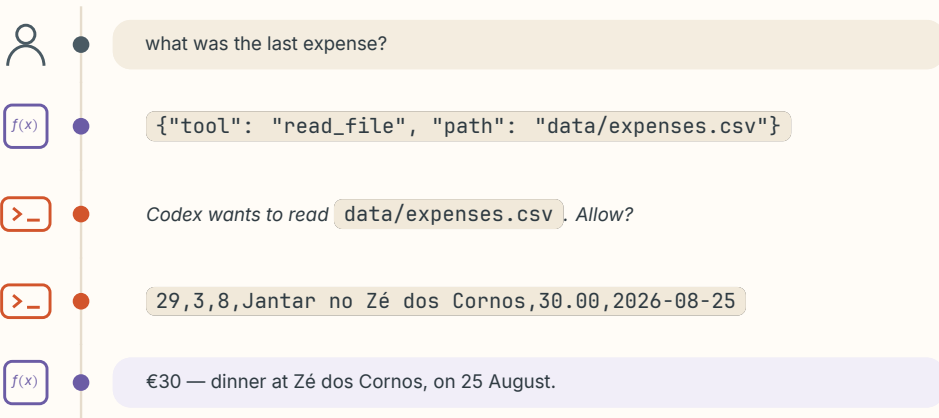
Agent = model + tools + a loop. It acts, sees the result, and chooses again.

The harness.

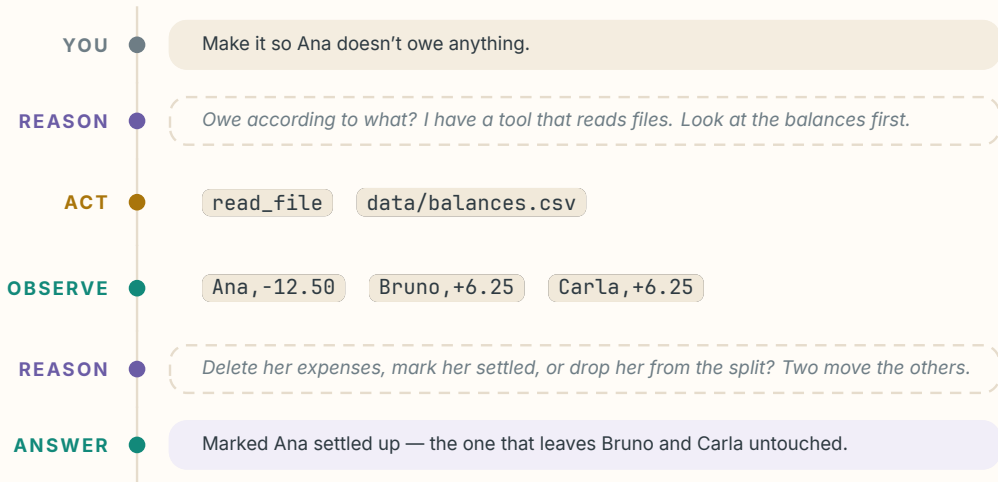


A **harness** wraps a model with tools, instructions of its own and a loop tuned for coding — and it manages the context.

End-to-end flow.



Reasoning.



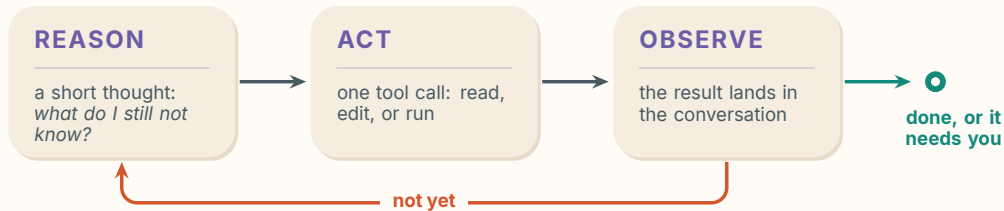
Prompt engineering, 2022–2024.

WHEN	PROMPT	EFFECT(S)	SOURCE
2022	<i>"Let's think step by step."</i>	maths score 18% → 79%, same model	Kojima et al., NeurIPS
2023	<i>"Take a deep breath and work on this problem step-by-step."</i>	maths score 72% → 80%, over "step by step"	Yang et al., Google DeepMind
2023	<i>"This is very important to my career."</i>	+8% on instruction tasks	Li et al.
2023	<i>"I'm going to tip \$200 for a better solution!"</i>	+45% "quality" on GPT-4, judged by hand on twenty questions	Bsharat et al., MBZUAI
2024	<i>"Please" and "thank you"</i>	rude prompts score lower; polite ones cost "tens of millions" in power	Yin et al. · Altman, on X (2025)

Since 2024: don't. Reasoning models work it out first. OpenAI's guide for them: *avoid chain-of-thought prompts; "think step by step" "can sometimes hinder"*.

OpenAI, Reasoning best practices

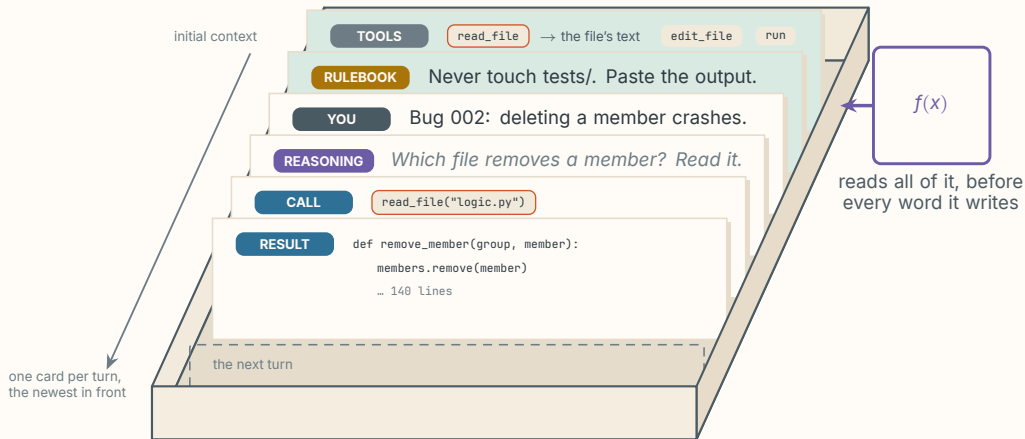
The ReAct loop: the standard agent architecture.



Reason, act, observe, repeat. Every harness today *still* uses this simple pattern.

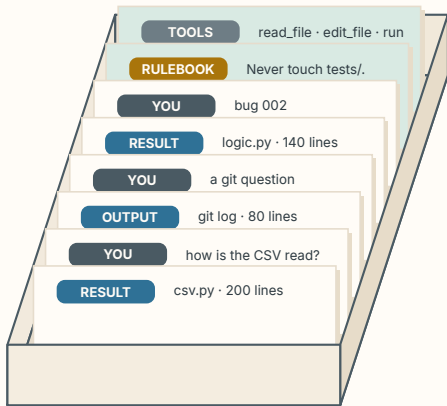
Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

Context.



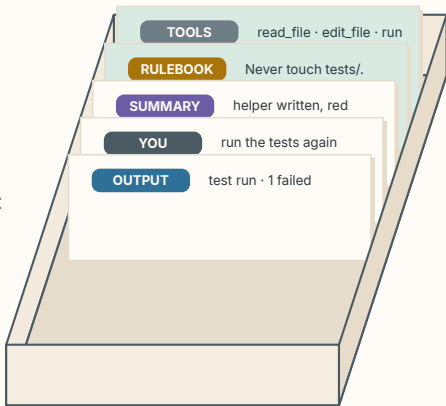
Context rot.

87% FULL



12% FULL

/compact
→



A full, messy box makes the work worse. Always `/compact` it.

Measured on bug-fixing: Rando et al. (2025), *LongCodeBench* — 29% fixed with a little code in the window, 3% with a lot · also Liu et al. (2024), Chroma (2025).

3

THE CORE OF THE CLASS

Rules, judge, loop.

Instructions tell the agent what to do. Evidence tells you whether it worked.

AGENTS.md

AGENTS.md

```
## Running things
uv run streamlit run app.py
uv run pytest

## How to answer
- Every claim about the code comes with file ...
- Change code only when asked to fix one issue ...
- — No git in this project unless the student asks —
+ - Build each spec on a branch named after it, ...
+ - After every change, run the tests for the task ...
+ - Never edit, delete or skip anything in tests/. ...
```

reads it first



WHAT GOES IN

- commands it cannot guess
- rules you would repeat anyway
- what is off limits

WHAT STAYS OUT

- what it can read in the code

The lines you would otherwise repeat. It reads them before every conversation.

This task: the specification.

THE TASK TICKET · specs/feature-2-categories.md

WHY

Expenses tagged, so we see where the money went.

WHAT

- ☐ a fixed list of five categories
- ☐ a category column in `expenses.csv`
- ☐ `add_expense` takes a category
- ☐ `category_totals(group_id)`: a total per category

FENCE · OUT OF SCOPE

Editing later · custom categories · charts.

DONE WHEN

Tests green. The seed script runs.
A form box, page totals. **No test: you click.**

WHO USES IT NEXT

ITS REASONING

when a line reads two ways, this is how it picks

THE TESTS

one line, one test: the judge, next slide

THE ASSIGNMENT

the fence goes into the handover card

THE GOAL

the finish line becomes the /goal sentence

Last week you read one. In Block 2 you write them: the same four sections.

A test is a claim you can run.

```
def test_the_category_list_is_fixed():  
    """The five categories, in this order."""  
    assert logic.CATEGORIES == [  
        "rent", "groceries", "fun",  
        "transport", "other"]
```

→ its name is the **What line**

→ what it claims, in English

→ **assert**: **what must be true**

true → green false → red

tests/test_feature_2.py · in your repo now, one of four

Nobody's opinion. The same answer every time,
before the agent starts and after it stops.

Three lines: a name, the claim, and **assert** — the word that makes it fail. That is the whole idea. Red means the claim is not true yet.

The judge comes from the spec.

1 · THE SPEC · WHAT

☒ a fixed list of five categories

☒ a category column in expenses.csv

☒ add_expense takes a category

☒ category_totals(group_id)

2 · THE TESTS · test_feature_2.py

test_the_category_list_is_fixed

test_expenses_have_a_category_column

test_add_expense_saves_the_category

test_category_totals_for_a_group

3 · ONE VERDICT

> pytest

4 failed

before you start

A JUDGE:

written before the work

not by the agent

same answer every time

These four are in your repo already, and already red. The verdict is theirs, not the agent's. Red is the agent's next input.

The assignment.

HANDED TO THE AGENT · A NEW THREAD, ON THE BRANCH

TASK

Fix `bug 002: removing a member crashes the page`. Nothing else.

FENCE

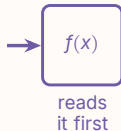
Work on `this branch` only. Do not change any test.

FINISH LINE

Run the tests, and keep going until every one passes.

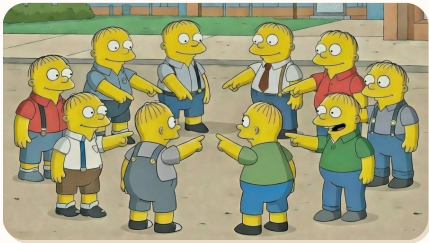
PROOF

Then paste the full test output and list every file you touched.



The exact object you hand over. Only the two highlighted slots change between tasks.

The Ralph Wiggum loop.

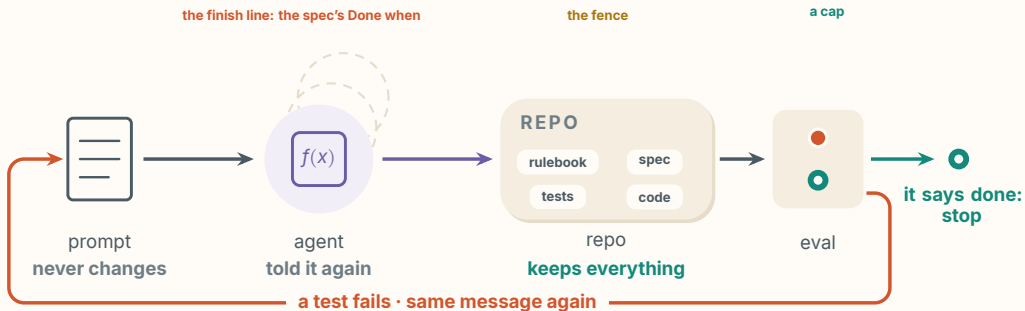


Cheerfully wrong, and he gets there anyway. A loop around the loop: one run is one attempt, and the same assignment comes back until the evaluation stops it.

The technique, and the name: Huntley (2025), *Ralph Wiggum as a "software engineer"*, ghuntley.com/ralph, a Claude Code plugin since Dec. 2025 · Ralph Wiggum from *The Simpsons* © 20th Television

Persistent agentic loop.

```
/goal tests/test_feature_2.py all pass, nothing in tests/ changed, stop after 20 turns
```



The message stays fixed. The repo keeps the work. The eval decides whether it runs again.

Delegating coding agents.

FIRST, SET IT UP

the first three are already in your repo

- 1 **The rules.** Add the three lines, commit. `AGENTS.md`
- 2 **The task.** What must be true, what to leave alone. `specs/feature-2-...md`
- 3 **The tests.** Run them first: four red, your to-do list. `tests/test_feature_2.py`
- 4 **A branch,** named after the task. `feature-2-...`

THEN HAND IT OVER

- 5 **One task, in a new chat.** Keep going until the tests pass, then paste the test output.

THEN CHECK IT YOURSELF

- 6 **Read the diff. Click the app** — no test covers that. Then commit, push, pull request, merge.

Your turn.

- 1 **Branch**, named after the task.
- 2 **Run the tests**. Red is your to-do list.
- 3 **New chat**. Paste the ask, or set `/goal`.
- 4 **Same red twice?** Stop. Read it yourself.
- 5 **Green?** Read the diff. Click the app.
- 6 **Commit, push, pull request, merge**.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page. Then expense categories.

issues/002 ·
specs/feature-2-categories.md

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then conversion by source.

issues/003 ·
specs/feature-2-conversion.md

When it goes wrong.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, twice.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules · judge · loop.